```
    {
      "filename" : "file2.txt",
      "filetype" :  "private"
    }
  ]
}
```

Once this document is successfully indexed, the final piece of the jigsaw is the search. Listing 4.31 shows the search query written to fetch documents. The criteria are emails with an attachment that has file1.txt and `private` as the filename and classification type, respectively. This combination doesn't exist, so the results must be empty, unlike in the case of an object where the data is searched across documents, leading to false positive results.

Listing 4.31 Fetching results matching a filename and classification

```
GET emails_nested/_search
{
  "query": {
    "nested": {
      "path": "attachments",
      "query": {
        "bool": {
          "must": [
            { "match": { "attachments.filename": "file1.txt" }},
            { "match": { "attachments.filetype":  "private" }}
          ]
        }
      }
    }
  }
}
```

A nested query to fetch data from nested fields

The path points to the name of the nested field.

Search clauses: must match file1.txt and private

This query searches for a file named file1.txt with a `private` classification, which doesn't exist. No documents are returned for this query, which is exactly what we expect. The classification of file1.txt is `confidential`, not `private`, so it doesn't match. When a `nested` type represents an array of inner objects, the individual object is stored and indexed as a hidden document.

The `nested` data type is good at honoring associations and relationships. If you ever need to create an array of objects, each of which must be treated as an individual object, the `nested` data type may become your friend.

Elasticsearch has no array types

While we are on the subject of arrays, interestingly, there is no `array` data type in Elasticsearch. However, we can set any field with more than one value, thus representing the field as an array. For example, a document with one name field can be changed from a single value to an array simply by adding a list of data values to the field: `"name": "John Doe"` to `"name": ["John Smith", "John Doe"]`.